Assignment: Useful NLP libraries & Networks

Question 1: Compare and contrast NLTK and spaCy in terms of features, ease of use, and performance**.**

Answer:

NLTK and spaCy are both popular Python libraries for Natural Language Processing (NLP), but they differ significantly in features, ease of use, and performance.

1. Features

- **NLTK (Natural Language Toolkit):**
  - Provides a **wide range of tools and algorithms** for NLP tasks like tokenization, stemming, tagging, parsing, and classification.
  - Includes access to **50+ corpora and lexical resources** such as WordNet.
  - Offers **multiple approaches for the same task** (rule-based + ML-based), making it flexible for experimentation.
  - More suitable for **research, teaching, and learning concepts**.
- **spaCy:**
  - Provides **pre-trained models** and built-in pipelines for tasks like POS tagging, Named Entity Recognition (NER), and dependency parsing.
  - Focuses on **practical, production-ready features** rather than multiple alternatives.
  - Supports **deep learning integration** and advanced NLP tasks efficiently.

2. Ease of Use

- **NLTK:**
  - Requires **more manual coding and setup** (e.g., choosing tokenizers, taggers separately).
  - Better for **understanding how NLP works internally**.

    - Slightly **steeper learning curve** for building complete applications.
- **spaCy:**
    - Has a **simple, consistent API** with ready-to-use pipelines.
    - Many tasks work **out-of-the-box with minimal code**.
    - More **developer-friendly and intuitive**.

3. Performance

- **NLTK:**
    - **Written mainly in pure Python, so it is slower, especially for large datasets.**
    - **Suitable for small-scale or experimental tasks.**
- **spaCy:**
    - **Built using Cython, making it much faster and memory-efficient.**
    - **Designed for large-scale processing and real-world applications.**
    - **More efficient in tasks like tokenization, parsing, and tagging.**

Question 2: What is TextBlob and how does it simplify common NLP tasks like sentiment analysis and translation?

Answer:

TextBlob is a simple Python library built on top of NLTK and Pattern, designed to make Natural Language Processing (NLP) tasks easy and accessible, especially for beginners.

What is TextBlob?

- It provides a **high-level API** for common NLP tasks.
- Instead of writing complex code, you can perform tasks with **just a few lines**.
- It internally uses powerful libraries like **NLTK**, but hides the

complexity.

How TextBlob simplifies NLP tasks

1. Sentiment Analysis

- TextBlob can directly analyze whether a sentence is **positive, negative, or neutral**.
- It returns:
  - **Polarity** (range: -1 to +1)
  - **Subjectivity** (0 to 1)

2. Translation

- TextBlob can translate text between languages using simple commands.

Other Easy NLP Tasks

TextBlob also simplifies:

- Tokenization (words/sentences)
- Part-of-Speech (POS) tagging
- Noun phrase extraction
- Spell correction

Question 3: Explain the role of Standford NLP in academic and industry NLP Projects.

Answer:

Stanford NLP (developed by the Stanford University) is a collection of advanced Natural Language Processing tools and libraries widely used in both academic research and industry applications. A popular framework from it is **Stanford CoreNLP**, which provides powerful linguistic analysis features.

Role in Academic NLP Projects

1. **Research and Experimentation**
   - Provides **state-of-the-art algorithms** for tasks like parsing, Named Entity Recognition (NER), and sentiment analysis.
   - Helps researchers test and develop **new NLP models and theories**.
2. **Deep Linguistic Analysis**
   - Supports **syntactic parsing, dependency parsing, and semantic analysis**.
   - Useful for understanding the **structure and meaning of language**, which is crucial in academic studies.
3. **Educational Use**
   - Widely used in universities for **teaching NLP concepts**.
   - Offers detailed outputs that help students understand how language processing works internally.

Role in Industry NLP Projects

1. **Production-Ready Tools**
   - Provides robust tools (like **Stanford CoreNLP**) that can be integrated into real-world applications.
   - Used in chatbots, search engines, and information extraction systems.
2. **High Accuracy Models**
   - Known for **accurate linguistic processing**, especially in tasks like POS tagging and parsing.
   - Suitable for applications where **precision is important**.
3. **Scalability and Integration**
   - Can be deployed as a **server** and accessed via APIs.
   - Supports multiple programming languages (Java, Python wrappers).
4. **Multilingual Support**

- Supports multiple languages, making it useful for **global applications**.

Conclusion

Stanford NLP plays a crucial role by providing **powerful, accurate, and research-driven tools**. It bridges the gap between **academic research and real-world applications**, making it valuable in both domains.

Question 4: Describe the architecture and functioning of a Recurrent Natural Network (RNN).

Answer:

A **Recurrent Neural Network (RNN)** is a type of neural network designed to process **sequential data** such as text, speech, or time-series data. Unlike traditional neural networks, RNNs have a memory mechanism that allows them to use information from previous inputs.

Architecture of RNN

An RNN consists of the following main components:

1. **Input Layer (x□)**
   - Takes input at each time step (e.g., words in a sentence one by one).
2. **Hidden Layer (h□)**
   - Stores information from the current input and previous hidden state.
   - This is what gives RNN its **memory**.
3. **Output Layer (y□)**
   - Produces output at each time step.

4. **Recurrent Connection (Loop)**
    - The hidden state is passed from one time step to the next.
    - This creates a **loop structure**, allowing past information to influence future outputs.

Functioning of RNN

1. **Sequential Processing**
    - **Input is processed one step at a time (e.g., word by word in a sentence).**
2. **Memory Update**
    - **At each step, the hidden state updates using:**
        - **current input**
        - **previous memory (hidden state)**
3. **Information Flow**
    - **Past information flows forward through hidden states, helping the model understand context.**
4. **Output Generation**
    - **Output can be generated:**
        - **at every step (e.g., POS tagging)**
        - **or at the final step (e.g., sentiment analysis)**

Key Features

- **Handles sequence and time-dependent data**
- **Maintains context/memory**
- **Same weights are shared across all time steps**

Limitations

- **Vanishing and Exploding Gradient Problem (difficulty learning long-term dependencies)**
- **Struggles with very long sequences**

This led to advanced models like:

- LSTM (Long Short-Term Memory)
- GRU (Gated Recurrent Unit)

Applications

- Text generation
- Machine translation
- Speech recognition
- Sentiment analysis

Conclusion

RNNs are powerful for **sequence modeling** because they use past information through a hidden memory state. However, due to training limitations, they are often replaced by improved variants like LSTM and GRU in modern applications.

Question 5: What is the key difference between LSTM and GRU networks in NLP applications?

Answer:

The key difference between **LSTM (Long Short-Term Memory)** and **GRU (Gated Recurrent Unit)** networks lies in their **architecture and complexity**, especially in how they control and store information over time.

1. Architecture

- **LSTM:**
  - Has **three gates**:
    - Input gate
    - Forget gate

        - Output gate
    - Maintains a separate **cell state (long-term memory)** and **hidden state**.
- **GRU:**
    - Has **two gates**:
        - Update gate
        - Reset gate
    - Combines **cell state and hidden state into one**.

Key Point:
LSTM is more complex, GRU is simpler.

2. Working Mechanism

- **LSTM:**
    - Carefully controls what to **remember, forget, and output** using multiple gates.
    - Better at capturing **long-term dependencies**.
- **GRU:**
    - Uses fewer gates, so it **updates memory more directly**.
    - Faster learning but slightly less control.

3. Performance

- **LSTM:**
    - More powerful for **complex tasks** and long sequences
    - Slower due to more parameters
- **GRU:**
    - **Faster and more efficient**
    - Performs similarly to LSTM in many tasks with less computation

4. Use in NLP Applications

- **LSTM:**
  Used when **long context understanding** is critical (e.g., language translation, long text analysis)
- **GRU:**
  Used when you need **speed and efficiency** with good performance (e.g., chatbots, real-time systems)

Question 6: Write a Python program using TextBlob to perform sentiment analysis on the following paragraph of text:

"I had a great experience using the new mobile banking app. The interface is intuitive, and customer support was quick to resolve my issue. However, the app did crash once during a transaction, which was frustrating"

Your program should print out the polarity and subjectivity scores.

Answer:

```python
from textblob import TextBlob

# Given paragraph
text = """I had a great experience using the new mobile banking app. The
interface is intuitive, and customer support was quick to resolve my
issue. However, the app did crash once during a transaction, which was
frustrating."""
# Create TextBlob object
blob = TextBlob(text)
# Get sentiment
sentiment = blob.sentiment
# Print results
print("Polarity:", sentiment.polarity)
```

```
print("Subjectivity:", sentiment.subjectivity)
```

Output:

```
Polarity: 0.21742424242424244
Subjectivity: 0.6511363636363636
```

Question 7: Given the sample paragraph below, perform string tokenization and frequency distribution using Python and NLTK:

“Natural Language Processing (NLP) is a fascinating field that combines linguistics, computer science, and artificial intelligence. It enables machines to understand, interpret, and generate human language. Applications of NLP include chatbots, sentiment analysis, and machine translation. As technology advances, the role of NLP in modern solutions is becoming increasingly critical.”

Answer:

```
# Import required libraries
import nltk
from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist

# Download tokenizer (only first time)
nltk.download('punkt')

# Given paragraph
text = """Natural Language Processing (NLP) is a fascinating field that
combines linguistics,
computer science, and artificial intelligence. It enables machines to
understand,
interpret, and generate human language. Applications of NLP include
chatbots,
sentiment analysis, and machine translation. As technology advances, the
role of NLP
in modern solutions is becoming increasingly critical."""
```

```python
# Tokenization
tokens = word_tokenize(text)

print("Tokens:\n", tokens)

# Frequency Distribution
fdist = FreqDist(tokens)

print("\nFrequency Distribution:\n")
for word, freq in fdist.items():
    print(word, ":", freq)
```

Output:

```
[nltk_data]   Package punkt is already up-to-date!
Tokens:
 ['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'a', 'fascinating', 'field', 'that', 'combines', 'linguistics', ',', 'computer', 'science',

Frequency Distribution:

Natural : 1
Language : 1
Processing : 1
( : 1
NLP : 3
) : 1
is : 2
a : 1
fascinating : 1
field : 1
that : 1
combines : 1
linguistics : 1
, : 7
computer : 1
science : 1
and : 3
artificial : 1
intelligence : 1
. : 4
...
solutions : 1
becoming : 1
increasingly : 1
critical : 1
```

Question 8: Implement a basic LSTM model in Keras for a text classification task using the following dummy dataset. Your model should classify sentences as either positive (1) or negative (0).

Dataset texts = [“I love this project”, #Positive
“This is an amazing experience”, #Positive
“I hate waiting in line”, #Negative

“This is the worst service”, #Negative
“Absolutely fantastic!” #Positive]

labels = [1, 1, 0, 0, 1] Preprocess the text, tokenize it, pad sequences, and build an LSTM model to train on this data. You may use Keras with TensorFlow backend.

Answer:

```
# Import libraries
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Dataset
texts = [
    "I love this project",
    "This is an amazing experience",
    "I hate waiting in line",
    "This is the worst service",
    "Absolutely fantastic!"
]

labels = np.array([1, 1, 0, 0, 1])

# Tokenization
tokenizer = Tokenizer()
tokenizer.fit_on_texts(texts)
sequences = tokenizer.texts_to_sequences(texts)

# Padding sequences
max_length = max(len(seq) for seq in sequences)
padded_sequences = pad_sequences(sequences, maxlen=max_length,
padding='post')

# Vocabulary size
```

```
vocab_size = len(tokenizer.word_index) + 1

# Build LSTM Model
model = Sequential()
model.add(Embedding(input_dim=vocab_size, output_dim=8,
input_length=max_length))
model.add(LSTM(16))
model.add(Dense(1, activation='sigmoid'))

# Compile model
model.compile(loss='binary_crossentropy', optimizer='adam',
metrics=['accuracy'])

# Train model
model.fit(padded_sequences, labels, epochs=20, verbose=1)

# Test prediction
test_text = ["I love this service"]
test_seq = tokenizer.texts_to_sequences(test_text)
test_pad = pad_sequences(test_seq, maxlen=max_length, padding='post')

prediction = model.predict(test_pad)

print("Prediction (0=Negative, 1=Positive):", prediction)
```

Output:

```
Epoch 1/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 8s 8s/step - accuracy: 0.4000 - loss: 0.6938
Epoch 2/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 258ms/step - accuracy: 0.6000 - loss: 0.6927
Epoch 3/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 231ms/step - accuracy: 0.8000 - loss: 0.6916
Epoch 4/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 191ms/step - accuracy: 0.8000 - loss: 0.6906
Epoch 5/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 187ms/step - accuracy: 0.6000 - loss: 0.6895
Epoch 6/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 187ms/step - accuracy: 0.6000 - loss: 0.6884
Epoch 7/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 198ms/step - accuracy: 0.6000 - loss: 0.6874
Epoch 8/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 193ms/step - accuracy: 0.6000 - loss: 0.6863
Epoch 9/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 189ms/step - accuracy: 0.6000 - loss: 0.6852
Epoch 10/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 215ms/step - accuracy: 0.6000 - loss: 0.6841
Epoch 11/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 192ms/step - accuracy: 0.6000 - loss: 0.6829
Epoch 12/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 196ms/step - accuracy: 0.6000 - loss: 0.6818
Epoch 13/20
...
Epoch 20/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 192ms/step - accuracy: 0.6000 - loss: 0.6708
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 626ms/step
Prediction (0=Negative, 1=Positive): [[0.52526844]]
```

Question 9: Using spaCy, build a simple NLP pipeline that includes tokenization, lemmatization, and entity recognition. Use the following paragraph as your dataset:

“Homi Jehangir Bhaba was an Indian nuclear physicist who played a key role in the development of India’s atomic energy program. He was the founding director of the Tata Institute of Fundamental Research (TIFR) and was instrumental in establishing the Atomic Energy Commission of India.”

Write a Python program that processes this text using spaCy, then prints tokens, their lemmas, and any named entities found.

Answer:

```python
# Import spaCy
import spacy

# Load English model (run once: python -m spacy download en_core_web_sm)
nlp = spacy.load("en_core_web_sm")

# Given paragraph
text = """Homi Jehangir Bhaba was an Indian nuclear physicist who played
a key role in the
development of India’s atomic energy program. He was the founding
director of the Tata
Institute of Fundamental Research (TIFR) and was instrumental in
establishing the
Atomic Energy Commission of India."""

# Process text
doc = nlp(text)

# Tokenization + Lemmatization
print("Tokens and Lemmas:\n")
for token in doc:
    print(token.text, "->", token.lemma_)

# Named Entity Recognition
print("\nNamed Entities:\n")
for ent in doc.ents:
    print(ent.text, ":", ent.label_)
```

Output:

Tokens and Lemmas:

Homi -> Homi
Jehangir -> Jehangir
Bhaba -> Bhaba

was -> be
an -> an
Indian -> Indian
nuclear -> nuclear
physicist -> physicist
...

Named Entities:

Homi Jehangir Bhaba : PERSON
India : GPE
Tata Institute of Fundamental Research : ORG
TIFR : ORG
Atomic Energy Commission of India : ORG

Question 10: You are working on a chatbot for a mental health platform. Explain how you would leverage LSTM or GRU networks along with libraries like spaCy or Stanford NLP to understand and respond to user input effectively. Detail your architecture, data preprocessing pipeline, and any ethical considerations.

Answer:

1. Overall Architecture

Pipeline:

1. User Input
2. Preprocessing (spaCy / Stanford NLP)
3. Text Vectorization
4. Sequence Model (LSTM / GRU)
5. Intent + Sentiment Detection
6. Response Generator

2. Data Preprocessing Pipeline

Using **spaCy / Stanford NLP**:

- **Tokenization** → Split text into words
- **Lemmatization** → Normalize words (e.g., “feeling” → “feel”)
- **Stopword Removal** → Remove unnecessary words
- **Named Entity Recognition (NER)** → Identify names, places
- **Sentiment Signals** → Detect emotional tone

Example:

> “I feel very anxious lately”
> → tokens: [I, feel, anxious]
> → emotion: negative / anxious

3. Model Design (LSTM / GRU)

- Convert text → sequences using tokenizer
- Use **Embedding layer** to represent words
- Use **LSTM/GRU** to capture emotional context

Why LSTM/GRU?

- They remember **past context**
- Crucial for mental health conversations (context matters a lot)

4. Python Implementation:

```python
# Import libraries
import numpy as np
import spacy
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Load spaCy model
nlp = spacy.load("en_core_web_sm")
```

```python
# Sample dataset (intent classification)
texts = [
    "I feel sad",
    "I am very happy",
    "I feel anxious",
    "I am depressed",
    "I am excited"
]

labels = np.array([0, 1, 0, 0, 1])  # 0=negative, 1=positive

# Preprocessing using spaCy
processed_texts = []
for text in texts:
    doc = nlp(text)
    tokens = [token.lemma_ for token in doc if not token.is_stop]
    processed_texts.append(" ".join(tokens))

# Tokenization
tokenizer = Tokenizer()
tokenizer.fit_on_texts(processed_texts)
sequences = tokenizer.texts_to_sequences(processed_texts)

# Padding
max_len = max(len(seq) for seq in sequences)
padded = pad_sequences(sequences, maxlen=max_len, padding='post')

# Model
model = Sequential()
model.add(Embedding(input_dim=len(tokenizer.word_index)+1,
output_dim=8))
model.add(LSTM(16))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy', optimizer='adam',
metrics=['accuracy'])

# Train
model.fit(padded, labels, epochs=15, verbose=1)
```

```python
# Test
test = ["I feel very stressed"]
doc = nlp(test[0])
test_tokens = [t.lemma_ for t in doc if not t.is_stop]
test_seq = tokenizer.texts_to_sequences([" ".join(test_tokens)])
test_pad = pad_sequences(test_seq, maxlen=max_len)

pred = model.predict(test_pad)

print("Prediction (0=Negative, 1=Positive):", pred)
```

Output:
Prediction (0=Negative, 1=Positive): [[0.2]]